

Tarea UD4 — Gestor de gastos con FreeSimpleGUI

Módulo: ESP — Entornos y Sintaxis en Python (5098)

Unidad: UD4 — Introducción a Frameworks

Enunciado

Construir una aplicación de escritorio con **FreeSimpleGUI** para registrar gastos personales. Los datos deben persistir en un fichero **JSON** y la lógica de negocio debe estar **en un módulo separado** de la GUI (mismo patrón que `tareas.py` / `app.py` del tema).

Estructura de cada gasto

```
{"fecha": "2026-05-02", "concepto": "Libro", "importe": 14.90, "categoria": "Ocio"}
```

Categorías fijas: Alimentación , Transporte , Ocio , Hogar , Otros .

Funcionalidades mínimas

1. **Añadir gasto:** fecha, concepto, importe (€), categoría. Validar:
 - Concepto no vacío.
 - Importe numérico y positivo.
2. **Eliminar** el gasto seleccionado.
3. **Listar** todos los gastos en un `Listbox` .
4. **Total gastado** visible y actualizado en pantalla.
5. **Filtro por categoría** (desplegable `combo` con las categorías + opción "Todas").
6. **Persistencia:** al arrancar se cargan los gastos del fichero; cada cambio se guarda automáticamente.

Entregable

Comprimir la carpeta del proyecto en un fichero **APELLIDO_NOMBRE_tarea4.zip** y subirlo a la plataforma. El ZIP debe contener:

```
APELLIDO_NOMBRE_tarea4/
├─ app.py           # GUI
├─ gastos.py        # módulo con la lógica y persistencia
├─ requirements.txt
└─ .gitignore
```

- NO incluir en el ZIP:
- gastos.json (se genera al ejecutar la aplicación)
 - La carpeta del entorno virtual (.venv/ , venv/ , etc.)
 - __pycache__/

Rúbrica

Aspecto	Peso	Qué se evalúa
Separación lógica / GUI	20%	gastos.py expone funciones puras; app.py solo orquesta la ventana. La lógica no importa FreeSimpleGUI.
Persistencia JSON	15%	Lectura y escritura correctas con json . El archivo se crea si no existe y se guarda tras cada cambio.
Tratamiento de excepciones	15%	try/except en la carga (FileNotFoundError , json.JSONDecodeError). Validaciones lanzan ValueError desde la lógica; la GUI las caza y muestra popup_error .
Validación de datos	10%	Concepto no vacío, importe convertible a float y > 0 . Mensajes de error claros.
Total y filtro funcionales	20%	El total refleja los gastos mostrados (respetando el filtro). El filtro actualiza la lista en tiempo real.
Uso correcto del event loop	10%	Estructura while True: read() ... , manejo de WIN_CLOSED , actualizaciones con ventana[key].update(...) .

Aspecto	Peso	Qué se evalúa
Calidad del código	10%	Nombres descriptivos, funciones cortas, f-strings y list/dict comprehensions donde aplique, sin código muerto.

Pistas

- Calcular el total: `sum(g["importe"] for g in gastos_filtrados)` .
- Para el filtro, `gastos.py` puede exponer `filtrar(gastos, categoria)` que devuelva la lista completa si la categoría es "Todas" .
- Para formatear una fila: `f"{fecha} | {categoria:<12} | {importe:>7.2f} € {concepto}"` .
- El `Combo` dispara eventos si se crea con `enable_events=True` .

Requisitos técnicos

- Python 3.10+
- `FreeSimpleGUI` (instalar con `pip install FreeSimpleGUI`)
- En Linux puede hacer falta instalar `python3-tk` a nivel de sistema si Tkinter no está disponible.